

Mastering Hashing in Java

Mastering Hashing in Java – Real Examples

Whether you're cracking coding interviews or designing scalable systems, **Hashing** is a core skill every Java developer must master.

Here's a mini-tutorial with real examples 📌

1. What is Hashing?

Hashing is a technique to convert input data into a fixed-size value (hash code), often used for:

- Fast data access (HashMap, HashSet)
 - Avoiding duplicates
 - Data integrity (checksums, cryptography)
-

2. Real Example: Detect Duplicates using HashSet

```
java

import java.util.*;

public class DuplicatesFinder {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 1};
        Set<Integer> seen = new HashSet<>();
        for (int num : arr) {
            if (!seen.add(num)) {
                System.out.println("Duplicate found: " + num);
            }
        }
    }
}
```

```
    }  
  }  
}
```

⚡ 3. Use Case: Word Frequency Counter (HashMap)

```
java  
  
import java.util.*;  
  
public class WordFrequency {  
    public static void main(String[] args) {  
        String text = "hello java hello world";  
        String[] words = text.split(" ");  
        Map<String, Integer> freq = new HashMap<>();  
  
        for (String word : words) {  
            freq.put(word, freq.getDefault(word, 0) + 1);  
        }  
  
        freq.forEach((word, count) ->  
            System.out.println(word + " → " + count));  
    }  
}
```

💡 4. Custom Objects in HashSet? Override hashCode() and equals()

```
java  
  
class Person {  
    String name;  
    Person(String name) { this.name = name; }  
  
    @Override
```

```

public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Person)) return false;
    return this.name.equals(((Person) o).name);
}

@Override
public int hashCode() {
    return name.hashCode();
}
}

public class CustomHashSet {
    public static void main(String[] args) {
        Set<Person> set = new HashSet<>();
        set.add(new Person("Alice"));
        set.add(new Person("Alice")); // Won't be added again

        System.out.println("Size: " + set.size()); // 1
    }
}

```

5. Secure Hashing (SHA-256) – for Passwords, Tokens

```

java

import java.security.*;

public class HashPassword {
    public static void main(String[] args) throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest("secret123".getBytes());

        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        System.out.println("Hashed: " + sb.toString());
    }
}

```

Summary

- ✓ Hashing boosts performance
- ✓ HashSet/HashMap are powerful tools
- ✓ Custom objects need `equals()` + `hashCode()`
- ✓ Secure hashing is key in backend security